Lecture 10 — Mock National Championship & Post-Mortem

> National Championship Prep · Prerequisites: Nationals Lectures 1–9 (and the Year Curriculum and Regional track) · Estimated time: 5 h contest + 1.5 h debrief, and a 1 h post-mortem presentation in the following session

1. Overview

This session is a full simulation of a national championship (ICPC North America Championship, a national olympiad final, or a Codeforces Div 1 round stretched to five hours). The problem set below is **original** and was written for this course. Every problem is solvable with techniques from this track and its prerequisites, but the statements never name the technique, and — as in real contests — the problems are **not** ordered by difficulty.

The second half of the lecture is about the **post-mortem**. Teams that improve between regionals and nationals are rarely the ones that learn the most new algorithms; they are the ones that systematically find out *why* they lost points and change one habit at a time. Section 5 gives a template you should fill in after this mock and every future one.

How to use these notes:

- **Before the contest**: students see only the problem statements handout (Section 3's "Statement" paragraphs, expanded with full I/O format by the instructor). Do not read the sketches.
- **During the debrief**: go through Section 3 in the order of the expected difficulty curve (A, E, F, C, K, B, H, J, D, G, I, L).
- **Afterwards**: upsolve. Two reference solutions (C and I) are given in Section 4; the rest are left as exercises with enough detail to implement.

2. Contest rules

- Teams of three, one computer, five hours, a printed team reference document (≤ 25 pages, made in Lecture 1).
- ICPC scoring: number solved, then penalty time = Σ over solved problems of (minutes until AC + 20 × rejected submissions on that problem).
- Scoreboard freezes for the last hour.
- Language: C++17 with `-O2`. Default 8 MB stack — deep recursion on $3 \cdot 10^5$-vertex graphs will crash; plan for it.

- All input is integral unless stated; output real numbers with absolute or relative error ≤ 10^{-6}.

3. The problem set

Ratings are estimates on the Codeforces scale.

	Problem	Main technique	Lecture	Rating
A	Lanterns	interval-cover greedy	Regional 03	1300
B	Recurring Revenue	matrix power, vector queries	N07	2100
C	Toll Bridges	bridges, bridge tree, LCA	N09	2000
D	Distance Band	centroid decomposition	N04	2300
E	Echo	suffix array + LCP / SAM	N06	1900
F	Corridor	convex hull + rotating calipers	N08	1900
G	Overtime Assignment	min-cost flow, convex edge costs	N05	2300
H	Twin Stations	binary search + 2-SAT	N09	2200
I	Xor Treasure Map	cycle space + xor basis	N07, N09	2300
J	Dice Tower	divide & conquer NTT	N07	2200
K	Lighthouse	angular sweep, two pointers	N08	2000
L	Warehouse Districts	DP + convex hull trick	N03	2400

A — Lanterns (≈1300)

Statement. A street is the segment [0, L] ($1 \le L \le 10^9$). There are $n \le 2 \cdot 10^5$ lanterns; lantern i at integer position p_i with radius r_i lights the closed segment $[p_i - r_i, p_i + r_i]$. Output the minimum number of lanterns to switch on so that every point of [0, L] is lit, or −1.

Solution. Sort lanterns by left end. Keep `cur` = the right end of the lit prefix [0, cur], starting at 0. While `cur < L`, scan all lanterns with left end ≤ cur (a pointer that only moves forward) and take the maximum right end `best`; if `best ≤ cur` the point just after cur cannot be lit, answer −1; otherwise `cur = best` and count one lantern. The exchange argument: any optimal solution's lantern covering the point just after cur can be replaced by the one reaching farthest right.

Complexity. O(n log n). **Traps:** lanterns entirely outside [0, L]; $p_i + r_i$ exceeding 32 bits; the loop condition (`cur < L`, not `≤`).

B — Recurring Revenue (≈2100)

Statement. Given $k \le 20$, coefficients $c_1..c_k$, constants d, e, and $a_0..a_{k-1}$. For $i \ge k$, $a_i = \sum_j c_j a_{i-j} + d \cdot i + e \pmod{10^9+7}$. Answer $q \le 2000$ queries: a_n for $n \le 10^{18}$.

Solution. State vector $v_i = (a_{i-1}, \ldots, a_{i-k}, i, 1)$ of size $k + 2 \le 22$; the transition T maps v_i to v_{i+1} (first row: $c_1..c_k$, d, e; the `i` row is $i + 1 = i + 1 \cdot 1$). Precompute $P_j = T^{(2^j)}$ for $j < 60$ (60 matrix products, $\sim 6 \cdot 10^5$ operations each). For a query, start from v_k and multiply the **vector** by P_j for each set bit of $n - k$: $O(60 \cdot 22^2)$ per query.

Why the trick matters. Computing T^n per query costs $2000 \cdot 60 \cdot 22^3 \approx 1.3 \cdot 10^9$ multiplications — too slow. Vector-times-matrix is 22 times cheaper. Note that powers of the same matrix commute, so the order of multiplying P_j into the vector does not matter.

C — Toll Bridges (≈2000)

Statement. A connected undirected graph with $n, m \le 3 \cdot 10^5$ (multi-edges allowed, no self-loops). $q \le 3 \cdot 10^5$ queries (u, v): how many edges are there whose removal (alone) disconnects u from v?

Solution. Such an edge must be a bridge (removing a non-bridge disconnects nothing), and a bridge separates u from v iff it lies on the bridge-tree path between their 2-edge-connected components. Compute bridges with lowlink (skipping the parent **edge id**), label components, build the bridge tree, and answer with depth[a] + depth[b] − 2·depth[lca(a, b)].

Complexity. $O(n + m + q \log n)$. **Traps:** multi-edges; recursion depth on a $3 \cdot 10^5$-vertex path (the reference solution uses an iterative DFS).

D — Distance Band (≈2300)

Statement. An unweighted tree with $n \le 2 \cdot 10^5$ vertices and integers $1 \le L \le R \le n$. Count unordered pairs $\{u, v\}$, $u \ne v$, with $L \le \text{dist}(u, v) \le R$.

Solution. Let $f(X)$ = number of unordered pairs at distance $\le X$ (with $u \ne v$); answer $f(R) - f(L - 1)$. Centroid decomposition: for the current piece with centroid c, collect depths of all vertices in the piece (c at depth 0). With cnt[d] and prefix sums P, the number of ordered pairs (x, y) with depth(x) + depth(y) $\le X$ is Σ_d cnt[d] · P[min($X - d$, maxDepth)] (over $d \le X$). This counts pairs through c but also pairs inside the same child subtree (whose real path does not pass through c) and pairs (x, x). Subtract the same quantity computed per child subtree (with depths still measured from c); this also removes every diagonal pair (x, x) except (c, c), so subtract 1 more for (c, c) (when $X \ge 0$), halve to get unordered pairs, and recurse into the children's pieces. Each level costs O(piece size), so the total is $O(n \log n)$.

Variant. Squaring the depth-count polynomial with NTT at each centroid gives the number of pairs at **every** distance in $O(n \log^2 n)$ — useful if L, R vary per query.

E — Echo (≈1900)

Statement. A lowercase string s, $n \le 2 \cdot 10^5$, and an integer $1 \le k \le n$. Output the maximum length of a substring that occurs at least k times in s (occurrences may overlap); output 0 if none.

Solution. For $k = 1$ the answer is n. Otherwise build the suffix array and LCP array; a substring occurring $\ge k$ times is a common prefix of k suffixes that are consecutive in sa, and the longest common prefix of sa[i..i+k−1] is min(lcp[i..i+k−2]). Answer = max over windows of $k - 1$ consecutive lcp values of the window minimum (monotonic deque, or a sparse table). SAM alternative: maximum `len(v)` over states with `occ(v) ≥ k`.

Complexity. $O(n \log n)$. **Trap:** $k = 1$ (empty window).

F — Corridor (≈1900)

Statement. $n \le 2 \cdot 10^5$ integer points, $|\text{coordinates}| \le 10^9$. Find the minimum width of an infinite straight strip (region between two parallel lines) containing all points.

Solution. The strip of minimum width containing a convex polygon has one boundary line flush with a polygon edge (rotate the strip; its width as a function of the angle is minimized at an edge direction, since between edge directions the width is a function of the form $A \cdot |\sin(\theta + \varphi)|$ with fixed support vertices, which is concave on the interval). Build the hull without collinear points, then for each edge (i, i+1) advance the antipodal pointer j while the triangle area grows, and take min of cross(e, h[j] − h[i]) / |e|.

Complexity. $O(n \log n)$. **Traps:** fewer than 3 hull vertices (all collinear ⇒ 0); `long long` overflow if you square cross products.

G — Overtime Assignment (≈2300)

Statement. $n \le 100$ workers and $m \le 100$ tasks. Worker i can do task j at cost $c_{ij} \ge 0$ (or not at all). Every task must be done by exactly one worker. A worker who does t tasks additionally pays $o_i \cdot t(t-1)/2$ (the k-th task costs $o_i \cdot (k-1)$ extra). Minimize the total, or report impossible.

Solution. Min-cost flow. Source → worker i: m parallel edges of capacity 1 with costs $0, o_i, 2o_i, \ldots, (m-1)o_i$. Worker i → task j: capacity 1, cost c_{ij}. Task → sink: capacity 1, cost 0. Send flow m (impossible if max flow < m). Because the parallel edge costs are non-decreasing (convex total cost), a min-cost flow never uses a more expensive parallel edge while a cheaper one is free (swapping them would reduce cost), so the flow cost equals the true cost.

Complexity. V ≈ 200, E ≈ $2 \cdot 10^4$, m augmentations of Dijkstra with potentials (all costs non-negative initially): about $100 \cdot E \log V$ — fast.

H — Twin Stations (≈2200)

Statement. n ≤ 1000 stations, 2 ≤ n. Station i must be built at position a_i or b_i on a line (integers, $|\cdot| \le 10^9$). Maximize the minimum distance between two built stations.

Solution. Binary search D ("can every pair be at distance ≥ D?"). Boolean x_i = "station i at a_i". For each pair i < j and each of the four choice combinations whose positions are closer than D, add the clause ¬(choice of i ∧ choice of j). The configuration exists iff the 2-SAT instance is satisfiable. A station with $a_i = b_i$ is fine (both literals give the same position). Answer: the largest feasible D (D = 0 is always feasible).

Complexity. $O(n^2 \log C)$: $\sim 2 \cdot 10^6$ clauses, $\sim 4 \cdot 10^6$ edges per check, ~31 checks. Reuse adjacency buffers and use an iterative Tarjan/Kosaraju to stay within time and stack.

I — Xor Treasure Map (≈2300)

Statement. A connected undirected graph, n, m, q ≤ $2 \cdot 10^5$, weights $0 \le w < 2^{60}$ (self-loops and multi-edges allowed). The value of a walk is the xor of the weights of the edges it traverses, counted with multiplicity. For each query (u, v) output the maximum value of a walk from u to v.

Solution. Fix a spanning tree and d[v] = xor from vertex 1 to v along it. Any walk from u to v has value d[u] ⊕ d[v] ⊕ (xor of some set of cycles), and every element of the cycle space is achievable (walk to the cycle, around it, and back — the access path cancels). The cycle space is spanned by the fundamental cycles d[a] ⊕ d[b] ⊕ w of non-tree edges. Insert them into one xor basis; answer each query with the greedy `maxXor(d[u] ⊕ d[v])`.

Complexity. O((m + q) · 60). Reference code in Section 4.2.

J — Dice Tower (≈2200)

Statement. n dice; die i has f_i faces with values $v_{i1}..v_{i}_\{f_i\} \in [0, 10^5]$ (values may repeat). $\Sigma f_i \le 2 \cdot 10^5$ and $S = \Sigma_i \max_j v_{ij} \le 2 \cdot 10^5$. For every t in [0, S], output the number of ways (one face per die) to get total t, mod 998244353.

Solution. Die i is the polynomial $P_i(x) = \Sigma_j x^{v_{ij}}$; we need ΠP_i. Multiply with balanced divide and conquer over the dice (split so that the degree sums of the halves are balanced), or repeatedly multiply the two polynomials of smallest degree from a priority queue. Each original coefficient takes part in O(log n) NTTs of total size O(S), so the total is $O(S \log^2 S)$. Multiplying left to right costs $\Theta(n \cdot S \log S)$ in the worst case.

K — Lighthouse (≈2000)

Statement. A lighthouse at the origin; $n \le 2 \cdot 10^5$ ships at integer points (not the origin, $|\text{coordinates}| \le 10^9$) with values $1 \le w_i \le 10^9$. The beam is a closed sector with apex at the origin, infinite radius, and angle α degrees (integer, $1 \le \alpha \le 359$). Output the maximum total value of ships inside one beam position. It is guaranteed that the answer does not change if α changes by 10^{-6} degrees.

Solution. Some optimal sector has a ship on its clockwise boundary. Compute θ_i with `atan2l`, sort, append copies with $\theta + 2\pi$, and sweep with two pointers: for each start i, extend j while $\theta_j \le \theta_i + \alpha$ (with a small ε, legitimate by the guarantee) and $j < i + n$, maintaining the window sum.

Complexity. O(n log n). **Traps:** forgetting the wrap-around; a window longer than n (counting ships twice) when α is large; values summing beyond 32 bits.

L — Warehouse Districts (≈2400)

Statement. $n \le 10^5$ positive integers $a_i \le 10^4$ and $k \le \min(n, 100)$. Split the array into exactly k non-empty contiguous districts minimizing Σ (district sum)^2.

Solution. Prefix sums S. dp[j][i] = min over $p < i$ of dp[j−1][p] + $(S_i - S_p)^2 = S_i^2$ + min_p $(-2S_p \cdot S_i + dp[j-1][p] + S_p^2)$. For fixed j this is a lower envelope of lines with slope $-2S_p$, added in order of strictly decreasing slope (S is strictly increasing because $a_i > 0$), queried at increasing $x = S_i$ — the monotone deque convex hull trick, amortized O(1) per operation, O(nk) total $\approx 10^7$. Alternative: the cost $(S_i - S_p)^2$ satisfies the quadrangle inequality, so the optimal split point is monotone and divide & conquer optimization gives O(k n log n).

Traps: intercepts reach $\sim 10^{18}$ and slopes $\sim 2 \cdot 10^9$, so the intersection comparison multiplies to $\sim 10^{27}$ — use `__int128` (or `long double` with care); only $p \ge j - 1$ are valid for layer j.

4. Reference solutions

Both compile with `g++ -std=c++17 -O2 -Wall` and were checked against brute force on hundreds of random small graphs (C: remove each edge and test connectivity; I: BFS over (vertex, xor value) states with small weights).

4.1 C — Toll Bridges

Input: `n m q`, m edges `u v` (1-indexed), q queries `u v`. One answer per line.

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp
// Mock Nationals, Problem C "Toll Bridges": for each query (u, v) print the number of
// edges whose removal disconnects u from v in a connected undirected multigraph.
// Bridges via iterative lowlink -> bridge tree -> answer = tree distance (via LCA).
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<vector<pair<int,int>>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v; --u; --v;
        adj[u].push_back({v, i}); adj[v].push_back({u, i});
    }
    // iterative DFS for tin/low (avoids stack overflow on long paths)
    vector<int> tin(n, -1), low(n), parEdge(n, -1), it(n, 0);
    vector<bool> bridge(m, false);
    int timer = 0;
    for (int s = 0; s < n; s++) {
        if (tin[s] != -1) continue;
        vector<int> st = {s};
        tin[s] = low[s] = timer++;
        while (!st.empty()) {
            int v = st.back();
            if (it[v] < (int)adj[v].size()) {
                auto [to, id] = adj[v][it[v]++];
                if (id == parEdge[v]) continue;
                if (tin[to] == -1) {
                    parEdge[to] = id; tin[to] = low[to] = timer++;
                    st.push_back(to);
                } else low[v] = min(low[v], tin[to]);
            } else {
                st.pop_back();
                if (!st.empty()) {
                    int p = st.back();
                    low[p] = min(low[p], low[v]);
                    if (low[v] > tin[p]) bridge[parEdge[v]] = true;
                }
            }
        }
    }
    // 2-edge-connected components
    vector<int> comp(n, -1);
    int C = 0;
    for (int s = 0; s < n; s++) {
        if (comp[s] != -1) continue;
        vector<int> st = {s}; comp[s] = C;
        while (!st.empty()) {
            int v = st.back(); st.pop_back();
            for (auto [to, id] : adj[v]) if (!bridge[id] && comp[to] == -1) { comp[to] =
C; st.push_back(to); }
        }
        C++;
    }
    // bridge tree + binary lifting LCA
    vector<vector<int>> tree(C);
    for (int v = 0; v < n; v++)
```

```
        for (auto [to, id] : adj[v]) if (bridge[id] && v < to) {
            tree[comp[v]].push_back(comp[to]); tree[comp[to]].push_back(comp[v]);
        }
    int LOG = 1;
    while ((1 << LOG) < C) LOG++;
    vector<vector<int>> up(LOG + 1, vector<int>(C, 0));
    vector<int> depth(C, 0), order = {0};
    vector<bool> seen(C, false); seen[0] = true;
    for (size_t i = 0; i < order.size(); i++) {           // BFS from component 0
        int v = order[i];
        for (int to : tree[v]) if (!seen[to]) {
            seen[to] = true; depth[to] = depth[v] + 1; up[0][to] = v;
order.push_back(to);
        }
    }
    for (int j = 1; j <= LOG; j++) for (int v = 0; v < C; v++) up[j][v] = up[j - 1][up[j
- 1][v]];
    auto lca = [&](int a, int b) {
        if (depth[a] < depth[b]) swap(a, b);
        for (int j = LOG; j >= 0; j--) if (depth[a] - (1 << j) >= depth[b]) a = up[j][a];
        if (a == b) return a;
        for (int j = LOG; j >= 0; j--) if (up[j][a] != up[j][b]) { a = up[j][a]; b =
up[j][b]; }
        return up[0][a];
    };
    while (q--) {
        int u, v; cin >> u >> v; --u; --v;
        int a = comp[u], b = comp[v];
        cout << depth[a] + depth[b] - 2 * depth[lca(a, b)] << "\n";
    }
}
```

4.2 I — Xor Treasure Map

Input: `n m q`, m edges `u v w` (1-indexed), q queries `u v`. One answer per line.

```
#include <bits/stdc++.h>
using namespace std;

// Mock Nationals, Problem I "Xor Treasure Map": connected undirected graph with edge
// weights < 2^60; for each query (u, v) print the maximum xor of a walk from u to v
// (edges may be repeated; each traversal xors the weight in).
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<vector<pair<int,long long>>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v; long long w; cin >> u >> v >> w; --u; --v;
        adj[u].push_back({v, w}); adj[v].push_back({u, w});
    }
    long long basis[60] = {};
    auto insert = [&](long long x) {
```

```
        for (int i = 59; i >= 0 && x; i--) {
            if (!(x >> i & 1)) continue;
            if (!basis[i]) { basis[i] = x; return; }
            x ^= basis[i];
        }
    };
    vector<long long> d(n, -1);                // xor from vertex 0 along the DFS tree
    d[0] = 0;
    vector<int> st = {0};
    while (!st.empty()) {
        int v = st.back(); st.pop_back();
        for (auto [to, w] : adj[v]) {
            if (d[to] == -1) { d[to] = d[v] ^ w; st.push_back(to); }
            else insert(d[v] ^ d[to] ^ w);      // cycle value (tree edges give 0)
        }
    }
    while (q--) {
        int u, v; cin >> u >> v; --u; --v;
        long long r = d[u] ^ d[v];
        for (int i = 59; i >= 0; i--) r = max(r, r ^ basis[i]);
        cout << r << "\n";
    }
}
```

5. The post-mortem

5.1 Immediately after the contest (30 minutes, before the solutions are shown)

Write the **raw timeline** while memory is fresh: for each event, the time, the person, and what happened ("1:07 Bob reads K, thinks angular sweep", "1:31 C WA on test 14", "2:10 keyboard idle 9 minutes"). Export the submission log from the judge. Do not analyse yet.

5.2 Per-problem table

Fill in one row per problem, including problems you never opened.

Problem	Status (AC time / attempts)	First read (who, when)	Idea found (when)	Coding time	Wrong runs and cause	Root cause category	Note
A							
B							
…							
L							

Root-cause categories (choose one primary per lost problem or wasted interval):

1. **Reading** — misread constraint/output format; missed a guarantee.
2. **Idea** — did not find the algorithm; or found a wrong one and did not prove it.
3. **Knowledge** — knew the idea but not the technique (e.g., had never coded min-cost flow).
4. **Implementation** — bug in code that was conceptually right.
5. **Edge case / limits** — overflow, n = 1, recursion depth, precision.
6. **Strategy** — wrong problem order, two people on one problem, sunk-cost on a hard problem, idle keyboard.

5.3 Team-level questions

- Which problems did we never read carefully, and how hard were they really?
- When was the keyboard idle, and why?
- For each WA: would a stress test or a careful re-read of the statement have caught it? How long would that have taken?
- **Counterfactual score:** using only ideas the team *had* during the contest, what was the best achievable result with perfect execution? The gap between this and the real score is the strategy/implementation gap; the gap to the full set is the knowledge/idea gap. They need different training.
- What did the scoreboard tell us at 1:00 and 2:30 that we ignored?

5.4 Action items

At most **three**, each concrete and checkable at the next mock. Good: "Re-read every unsolved statement at the 2:00 mark", "Add an iterative lowlink to the notebook and test it on a 10^6 path", "Whoever gets the second WA on a problem writes a brute force before resubmitting". Bad: "Be more careful", "Practise more geometry".

5.5 Individual reflection (each member, 10 minutes)

- One problem I should have solved and why I did not.
- One technique to drill this week (with 3 concrete practice problems).
- One habit to keep.

5.6 Notebook diff

List every piece of code you wished you had: missing templates (e.g., D&C NTT driver, 2-SAT with self-test), wrong ones, and ones that were too hard to find. Update and re-print the notebook within the week.

6. Common mistakes in mock contests

- **Reading in letter order.** In this set E, F, C and K are among the five easiest; teams that read linearly often find them after B and D have eaten an hour.
- **No division of the set.** Split the problems among the three members for the first read, then report one-line summaries.
- **Coding before the complexity check.** Problem B's per-query matrix power or J's left-to-right multiplication both "work" on samples and TLE.
- **Silent duplication.** Two people solving the same problem in their heads for 40 minutes.
- **No brute force after repeated WA.** The second wrong answer on a problem should trigger a stress test (Year week 35).
- **Treating the mock as a test, not a measurement.** The score matters far less than the post-mortem; teams that skip Section 5 do not improve.

7. Practice problems

Real problems closely related to each mock problem — upsolve the mock first, then these:

- A — Codeforces 1066B *Heaters* — https://codeforces.com/problemset/problem/1066/B
- B — Codeforces 1182E *Product Oriented Recurrence* — https://codeforces.com/problemset/problem/1182/E
- C — Codeforces 1000E *We Need More Bosses* — https://codeforces.com/problemset/problem/1000/E
- D — Codeforces 161D *Distance in Tree* — https://codeforces.com/problemset/problem/161/D and 293E *Close Vertices* — https://codeforces.com/problemset/problem/293/E
- E — CSES 2106 *Repeating Substring* — https://cses.fi/problemset/task/2106
- F — UVa 10173 *Smallest Bounding Rectangle* (calipers, one step further)
- G — Codeforces 277E *Binary Tree on Plane* — https://codeforces.com/problemset/problem/277/E
- H — POJ 2296 *Map Labeler* — http://poj.org/problem?id=2296
- I — Codeforces 845G *Shortest Path Problem?* — https://codeforces.com/problemset/problem/845/G
- J — Codeforces 1251F *Red-White Fence* — https://codeforces.com/problemset/problem/1251/F
- K — Codeforces 1284E *New Year and Castle Construction* — https://codeforces.com/problemset/problem/1284/E
- L — Codeforces 311B *Cats Transport* — https://codeforces.com/problemset/problem/311/B and 321E *Ciel and Gondolas* — https://codeforces.com/problemset/problem/321/E

Full sets for the next simulations (pick ones no team member has seen):

- ICPC North America Championship problem sets — available on Kattis (search the problem source "ICPC North America Championship") — https://open.kattis.com/

- ICPC Northern Eurasia (NERC) finals on Codeforces, e.g. 2023–2024 NERC — https://codeforces.com/contest/1912
- NWERC, SWERC, CERC sets in the Codeforces Gym — https://codeforces.com/gyms

8. Further reading

- Lecture 1 of this track (training system, team roles, notebook) — re-read it before writing your post-mortem.
- Year week 35 (contest strategy, stress testing) and Regional 01 (contest anatomy, triage).
- ICPC World Finals problem analyses and NAC editorials published by the judges each year — compare your post-mortem categories with what the judges expected to be hard.
- Next: World Finals track, Lecture 1 — WF format, problem archetypes, and five-hour strategy.